Graph Theory

Lecture 7

Spanning Trees, MSTs & Network Optimization

Counting • Matrix-Tree Theorem • MST Properties • Algorithms • Applications • Prufer Codes

From Tree Structure to Network Backbones

Lecture 5 established that a tree is minimally connected:

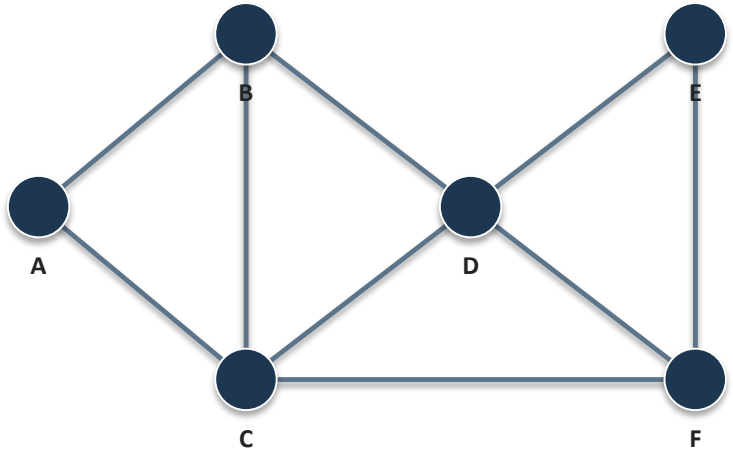
- n vertices → exactly n−1 edges
- remove any edge → disconnected
- add any missing edge → exactly one cycle

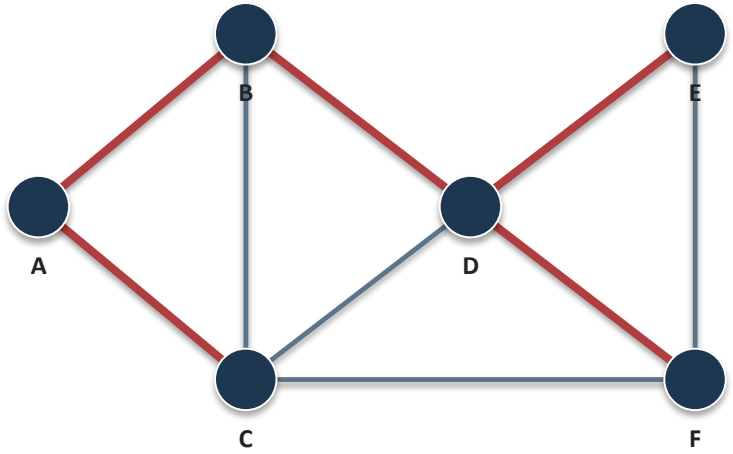
Today we start with a connected graph containing redundant links.

Question: Which edges can we keep so that every vertex remains connected but all cycles disappear?

Answer: choose a spanning tree.

Spanning Tree = Minimal Connected Backbone





Original graph: cycles + redundancy

One possible spanning tree: 6 vertices, 5 edges

One Graph Can Have Many Spanning Trees

A spanning tree must contain every vertex, but the edge choice is generally not unique.

Different spanning trees may have:

- the same number of edges — always n−1;
- different total cost in a weighted graph;
- different path lengths and operational characteristics.

This leads to two distinct questions:

How many spanning trees exist?

Which spanning tree is best?

Counting Spanning Trees: Cayley's Formula

$$\tau(K_n) = n^{n-2}$$

$\tau(G)$ denotes the number of spanning trees of G.

For complete graphs:

$\tau(K_3)=3$

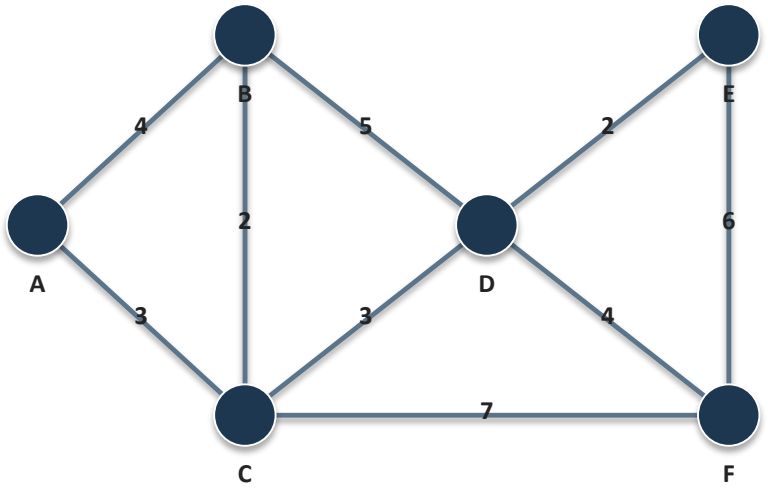
$\tau(K_4)=4^2=16$

$\tau(K_5)=5^3=125$

Even small dense graphs can therefore contain many possible tree backbones.

Cayley's formula is specific to complete graphs. What about an arbitrary graph?

Weighted Graphs: Not All Spanning Trees Are Equal



Interpret an edge weight as:

- installation cost
- distance / cable length
- latency
- construction effort

For a spanning tree T:

$w(T) = \Sigma\, w(e)$

An MST is a spanning tree with minimum total weight.

MST Theory: Why Greedy Choices Can Be Correct

Prim's and Kruskal's algorithms are not merely recipes.

Their correctness is supported by structural properties of minimum spanning trees.

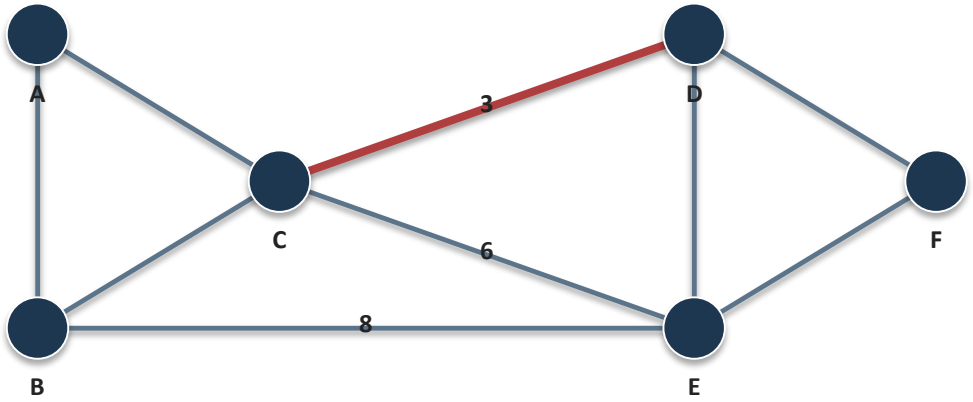
Two central ideas:

CUT PROPERTY
Which edge is safe to include?

CYCLE PROPERTY
Which edge is safe to exclude?

These properties are more fundamental than memorizing algorithm steps.

Cut Property: A Safe Edge for an MST



Imagine the cut {A,B,C} | {D,E,F}.

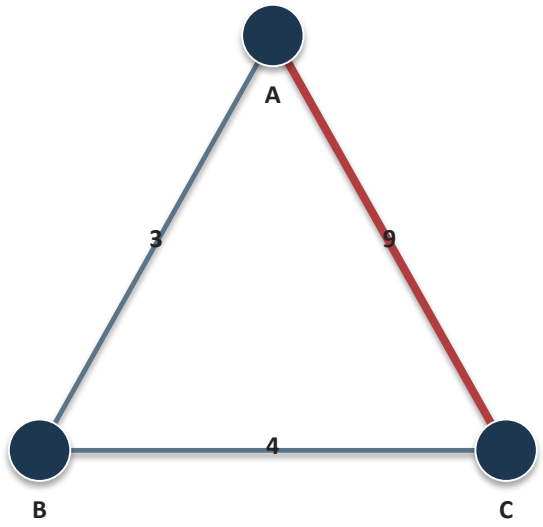
Edges crossing it have weights 3, 6 and 8.

The lightest crossing edge (weight 3) is safe for some MST.

Cut Property:

> a minimum-weight edge crossing a cut can be included in an MST.

Cycle Property: A Heavy Edge Can Be Rejected



The cycle has edge weights 3, 4 and 9.

If the weight-9 edge were used, replacing it with a lighter cycle edge would reduce cost.

Cycle Property:

> if an edge is strictly heavier than every other edge in a cycle, it cannot belong to an MST.

This is the exclusion counterpart of the cut property.

When Is the MST Unique?

If all edge weights are distinct, the MST is unique.

Important: distinct edge weights are sufficient, but not necessary.

Repeated weights do not automatically imply multiple MSTs.

Multiple MSTs occur when alternative safe choices can produce different trees with the same minimum total weight.

Concept question:

Can two different MSTs have exactly the same total weight? Yes.

Kruskal Revisited: Theory Behind the Algorithm

Kruskal's idea:

- consider edges from smallest to largest weight;
- add an edge if it does not create a cycle;
- stop after n−1 edges.

Advanced interpretation:

- each accepted edge is a safe edge joining components;
- cycle-forming edges are rejected because they are unnecessary for connectivity.

Efficient implementations use Disjoint Set Union (Union-Find) to detect component merges.

Prim Revisited: Growing Across a Cut

Prim's idea:

- start from any vertex;
- maintain one growing tree;
- repeatedly choose the lightest edge from the current tree to a new vertex.

Why is that choice safe?

- the current tree defines one side of a cut;
- Prim chooses a minimum-weight edge crossing that cut;
- therefore the Cut Property justifies the greedy step.

Prim vs. Kruskal: Same Goal, Different Growth

KRUSKAL

- Starts with a forest
- Chooses cheap edges globally
- Merges components
- Cycle detection is explicit
- Natural link to Union-Find

PRIM

- Starts with one vertex
- Grows one tree
- Chooses cheapest outgoing edge
- Cut is explicit
- Natural link to priority queues

Both exploit MST structural properties — not arbitrary greedy choices.

Where Are Spanning Trees and MSTs Used in Real Life?

Fiber / Telecom
Vertices: routers or sites
Weight: link cost
Use: minimum-cost backbone

Road / Rail
Vertices: cities
Weight: construction cost
Use: connect locations cheaply

Power Distribution
Vertices: substations
Weight: line cost
Use: radial distribution structure

Water / Gas
Vertices: supply points
Weight: pipe length
Use: basic low-cost network

Computer Networks
Vertices: switches
Weight: links
Use: loop-free spanning topology

Data Clustering
Vertices: data points
Weight: distance
Use: cluster structure from MST

Beyond Infrastructure: Broader Uses

Other uses include:

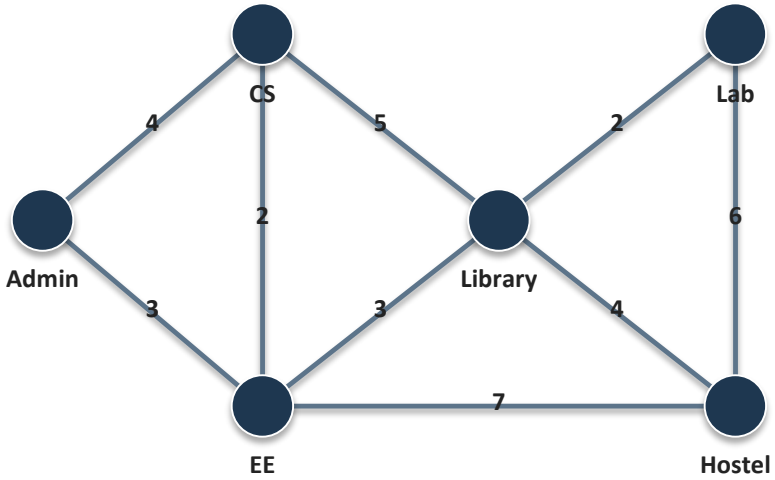
- image segmentation and computer vision;
- VLSI / PCB routing and wiring optimization;
- approximation and heuristic algorithms;
- hierarchical clustering and pattern analysis.

But remember:

An MST optimizes the chosen edge weight.

It does NOT automatically optimize reliability, capacity or delay.

Integrated Network Design Challenge



A university wants to connect all six buildings with fiber.

1. Find an MST and its total cost.
2. Which MST edges become bridges?
3. What happens if any one MST edge fails?
4. Add one extra edge that would improve resilience.
5. Explain the cost-versus-reliability trade-off.

The Design Trade-Off

MST Objective	But every MST is a tree	Engineering Objective
Minimum total connection cost	Every edge is a bridge	Balance cost with redundancy

The mathematically cheapest connected network may be operationally fragile.

Real network design often starts with an MST-like backbone, then adds selected redundant links.

Why Prüfer Codes?

A labeled tree on n vertices can be represented uniquely by a sequence of length n−2.

Every sequence of length n−2 over labels {1,…,n} corresponds to exactly one labeled tree.

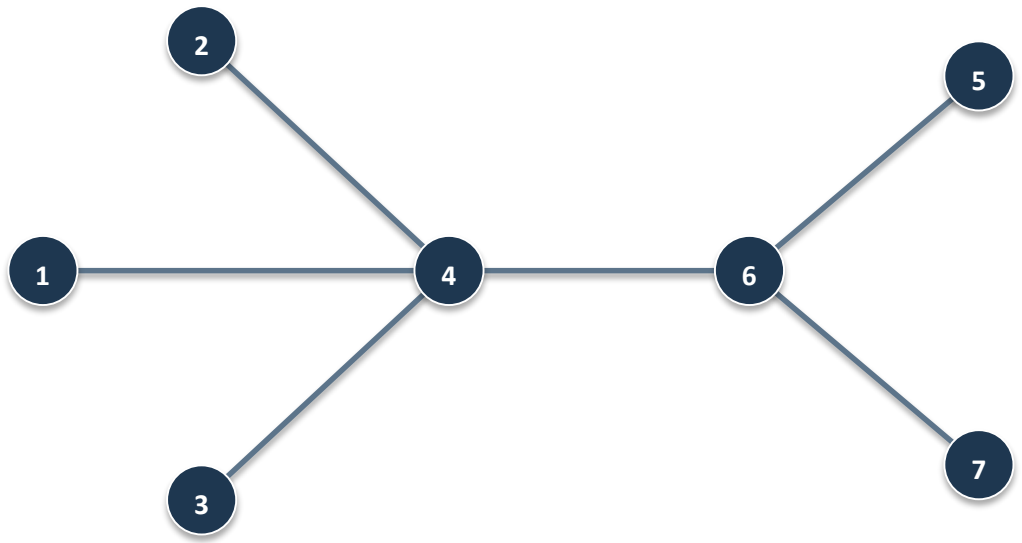
Therefore:

Labeled trees ↔ Prüfer sequences

This explains the n−2 in Cayley's formula and also provides a compact representation of trees.

Sequence length = n − 2

Encoding: Tree → Prüfer Code



Repeat until two vertices remain:

1. Choose the leaf with the smallest label.
2. Record its neighbor.
3. Delete that leaf.

For the tree shown:

1→4, 2→4, 3→4, 4→6, 5→6

P(T) = (4, 4, 4, 6, 6)

What the Code Tells Us Without Decoding

deg(v) = 1 + frequency of v in the Prüfer code

For P(T)=(4,4,4,6,6):

- 4 occurs 3 times → deg(4)=4
- 6 occurs 2 times → deg(6)=3
- 1,2,3,5,7 do not occur → each has degree 1

Hence a label is absent from the Prüfer code exactly when that vertex is a leaf.

Check: degree sum = 12 = 2(7−1).

Decoding: Prüfer Code → Tree

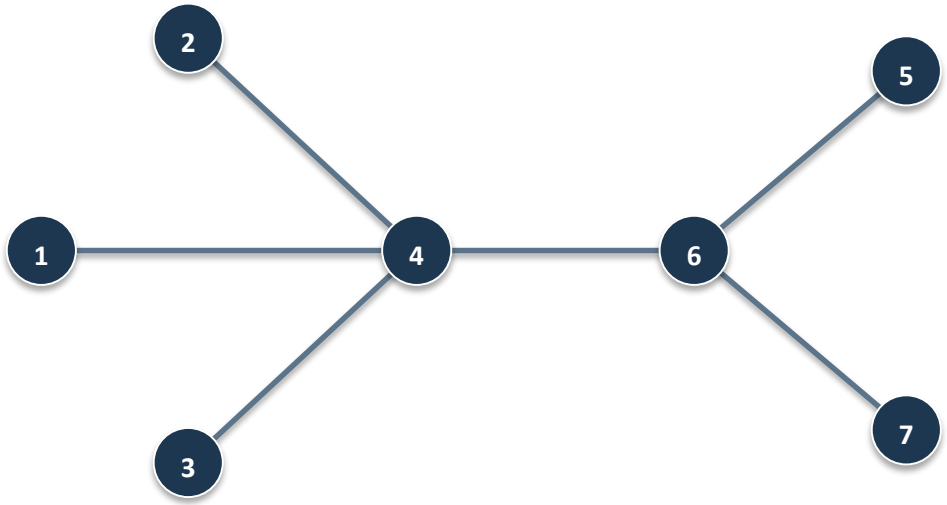
Given P=(4,4,4,6,6), n=5+2=7.

At each step:

- choose the smallest label not appearing in the current sequence;
- connect it to the first sequence entry;
- delete that first entry and repeat.

Edges obtained:

1–4, 2–4, 3–4, 4–6, 5–6; finally connect 6–7.



Encoding and decoding are inverse operations.

Prüfer Codes and Cayley's Formula

A tree on n labeled vertices corresponds to one sequence of length n−2.

Each of the n−2 positions has n possible labels.

Hence the number of possible Prüfer sequences is:

$$n \times n \times \cdots \times n = n^{n-2}$$

Therefore the number of labeled trees on n vertices is n^(n−2).

Example: K_5 has $5^3 = 125$ spanning trees.

Advanced Application: Count Trees with a Given Degree

How many labeled trees on {1,2,3,4,5,6} have deg(1)=3?

Prüfer sequence length = 4.

deg(1)=3 ⇒ label 1 must appear exactly 2 times.

Choose those two positions: C(4,2).

The remaining two positions each have 5 choices: {2,3,4,5,6}.

$$C(4,2) \times 5^2 = 6 \times 25 = 150$$

Concept Challenge & Why It Matters

P = (3, 3, 5, 2, 5)

Without reconstructing the tree:

- How many vertices?
- Find the degree of every vertex.
- Which vertices are leaves?

Verify using the Handshaking Lemma.

Uses of Prüfer codes

- Compact labeled-tree representation
- Random tree generation
- Enumeration and combinatorial counting
- Network-topology optimization
- Evolutionary/genetic algorithms

Key idea: frequency gives degree; absence gives leaves.

Concept Check

1. Does every connected graph have a spanning tree?
2. Does every spanning tree of the same graph have the same weight?
3. Why can the heaviest edge of a cycle sometimes be rejected?
4. If all edge weights are distinct, what can we say about the MST?
5. Why does Prim's greedy choice relate to the Cut Property?
6. Why can an MST be a poor design if fault tolerance is critical?

Lecture Summary

A spanning tree is a minimal connected backbone of a connected graph.

Cayley's formula counts spanning trees of complete graphs.

The Matrix-Tree Theorem counts spanning trees of arbitrary graphs using the Laplacian.

MSTs optimize total edge weight.

Cut and cycle properties explain safe inclusion and exclusion choices.

Prim and Kruskal are greedy algorithms grounded in those structural properties.

MSTs appear in infrastructure, networks, clustering and optimization — but minimum cost is not the same as maximum reliability.